In [4]:
```
import sys
import keyword
import operator
from datetime import datetime
import os
```

Keywords

Keywords are the reserve words in python and can't be used as an identifier

In [5]:
```
print(keyword.kwlist) # List all Python Keywords
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continu
e', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import',
'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'wit
h', 'yield']
```

In [6]:
```
len(keyword.kwlist) # Python contains 35 keywords
```

Out[6]:
```
35
```

Identifiers

An identifier is a name given to entities like class, functions, variables , etc. It helps to differenciate one entity from another.

In [8]:
```
1var = 10 # Identifier can't start with a digit
```

```
  Cell In[8], line 1
    1var = 10 # Identifier can't start with a digit
    ^
SyntaxError: invalid decimal literal
```

In [9]:
```
val2@ = 35 # Identifier cant use special symbols
```

```
  Cell In[9], line 1
    val2@ = 35 # Identifier cant use special symbols
          ^
SyntaxError: invalid syntax
```

In [10]:
```
import = 125 # Keywords can't be used as identifiers
```

```
  Cell In[10], line 1
    import = 125 # Keywords can't be used as identifiers
           ^
SyntaxError: invalid syntax
```

In [ ]:
```
" " "
Correct way of defining an identifier
(Identifiers can be a combination of letters in lowercase (a to z) or uppercase
" " "


val2 = 10
```

In [12]:
```
val_ = 99
```

Comments in Python

Comments can be used to explain the code for more readbility

In [13]:
```
# Single line comment
val1 = 10
```

In [14]:
```
# Multiple
# Line
# comment
val1 = 10
```

In [15]:
```
'''
Multiple
line
comment
'''
val1 = 10
```

In [16]:
```
"""
Multiple
line
comment
"""
val1 = 10
```

Statements

Instructions that a Python interpreter can execute

In [18]:
```
p = 20 #create an integer object with value 20 and assigns the variable p to p
q = 20 #create new reference q which will point to value 20 . p & q will be poir
r = q #variable r will also point to same location where p & q are pointin
p , type(p), hex(id(p)) # Variable P is pointing to memory location
```

Out[18]:
```
(20, int, '0x7ffe3106a608')
```

In [19]:
```
q , type(q), hex(id(q))
```

Out[19]:
```
(20, int, '0x7ffe3106a608')
```

In [20]:
```
r , type(r), hex(id(r))
```

Out[20]:
```
(20, int, '0x7ffe3106a608')
```

In [21]:
```
p = 20
p = p + 10 # Variable Overwriting
p
```

Out[21]:
```
30
```

Variable Assignment

In [23]:
```
intvar = 10 # Integer Variable
floatvar = 2.57 # Float Variable
strvar = "Python Lnguage" # String variable

print(intvar)
```

```
print(floatvar)
print(strvar)
```

```
10
2.57
Python Lnguage
```

Multiple Assignments

In [24]:
```
intvar , floatvar , strvar = 10, 2.57, "Python Lnguage"  # Using commas to separate
print(intvar)
print(floatvar)
print(strvar)
```

```
10
2.57
Python Lnguage
```

In [25]:
```
p1 = p2 = p3 = p4 = 44 # All variables are pointing to same value
print(p1,p2,p3,p4)
```

```
44 44 44 44
```

Data Types

Numeric

In [26]:
```
val1 = 10 # Integer data type
print(val1)
print(type(val1)) # type of  object
print(sys.getsizeof(val1)) # size of integer in bytes
print(val1, " is integer?" , isinstance(val1, int)) #val1 is an instance of int
```

```
10
<class 'int'>
28
10  is integer? True
```

In [28]:
```
val2  = 92.78 # Float data type
print(val2)
print(type(val2)) # type of object
print(sys.getsizeof(val2)) # size of float object in bytes
print(val2, "is float ?" , isinstance(val2 ,  float)) # val2 is an instance of float
```

```
92.78
<class 'float'>
24
92.78 is float ? True
```

In [29]:
```
val3 = 25 + 10j # Complex data type
print(val3)
print(type(val3)) # type of object
print(sys.getsizeof(val3)) # size of float object in bytes
print(val3, " is complex?" , isinstance(val3, complex)) # val3 is an instance of
```

```
(25+10j)
<class 'complex'>
32
(25+10j)  is complex? True
```

In [30]:
```
sys.getsizeof(int())
```
Out[30]: 28

In [31]:
```
sys.getsizeof(float())
```
Out[31]: 24

In [32]:
```
sys.getsizeof(complex())
```
Out[32]: 32

Boolean

Boolean data type can have only two possible value True or False

In [35]:
```
bool1 = True
```

In [36]:
```
bool2 = False
```

In [37]:
```
print(type(bool1))
```
```
<class 'bool'>
```

In [38]:
```
print(type(bool2))
```
```
<class 'bool'>
```

In [39]:
```
isinstance(bool1, bool)
```
Out[39]:
```
True
```
In [40]:
```
bool(0)
```
Out[40]:
```
False
```
In [41]:
```
bool(1)
```
Out[41]:
```
True
```
In [42]:
```
bool(None)
```
Out[42]:
```
False
```

Strings

String Creation

In [43]:
```
str1 = "HELLO PYTHON"

print(str1)
```
```
HELLO PYTHON
```

In [44]:
```
mystr = 'Hello World' # Define string using double quotes
print(mystr)
```

```
Hello World
```

In [45]:
```
mystr = '''Hello
            world'''
print(mystr)
```

```
Hello
            world
```

In [46]:
```
mystr = """Hello
            world"""
print(mystr)
```

```
Hello
            world
```

In [47]:
```
mystr = ('Happy'
         'Monday'
          'Everyone')
print(mystr)
```

```
HappyMondayEeveryone
```

In [48]:
```
mystr2 = 'Woohoo '
mystr2 = mystr2*5
mystr2
```

Out[48]:
```
'Woohoo Woohoo Woohoo Woohoo Woohoo '
```

In [49]:
```
len(mystr2) # Length of string
```
Out[49]:
```
35
```

String Indexing

In [50]:
```
str1
```
Out[50]:
```
'HELLO PYTHON'
```

In [51]:
```
str1[0] # First character in string "str1"
```
Out[51]:
```
'H'
```

In [54]:
```
str1[len(str1)-1]
```
Out[54]:
```
'N'
```

In [55]:
```
str1[6]
```
Out[55]:
```
'P'
```

In [56]:
```
str1[5]
```
Out[56]:
```
' '
```

String Slicing

In [57]:
```
str1[0:5] # String slicing - Fetch all chatercters from 0 to 5 index location exc
```

Out[57]:
```
'HELLO'
```

In [58]:
```
str1[6:12] # string slicing - Retrive all characters between 0 - 12 index lac e
```

Out[58]:
```
'PYTHON'
```

In [59]:
```
str1[-4:] # Retrive last four characters of the string
```

Out[59]:
```
'THON'
```

In [60]:
```
str1[-6:] # Retrive  last  six characters of the string
```

Out[60]:
```
'PYTHON'
```

In [61]:
```
str1[:4] # Retrive first four characters of the string
```

Out[61]:
```
'HELL'
```

In [62]:
```
str1[:6] # Retrive first six characters of the string
```

Out[62]:
```
'HELLO '
```

Update & Delete String

In [63]:
```
str1
```

Out[63]:
```
'HELLO PYTHON'
```

In [65]:
```
#String are immulate which means elements of a string cannot be changed once the
str1[0:5] = 'HOLAA'
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[65], line 2
      1 #String are immulate which means elements of a string cannot be changed once the
----> 2 str1[0:5] = 'HOLAA'

TypeError: 'str' object does not support item assignment
```

In [67]:
```
del str1 # Delete a string
print(str1)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[67], line 2
      1 del str1 # Delete a string
----> 2 print(str1)

NameError: name 'str1' is not defined
```

String Concatenation

In [68]:
```
# String concatenation
s1 = "Hello"
s2 = "Asif"
s3 = s1 = s2
print(s3)
```

```
Asif
```

In [ ]: